

IoT-Enabled Smart Water-Level Monitoring Bottle Using Ultrasonic Sensing and ESP32 Wireless Connectivity

Ch. Y.V.S. Srinivas, J. Hrusheekesh, M. Nishith Chowdary, P. Varshith Reddy, S. Mahamood Hussain, and B. Vijay Reddy

Department of Computer Science and Engineering (IoT)

Malla Reddy Deemed University

Hyderabad, Telangana 500100, India

nishith374@gmail.com

Abstract—Portable water-level awareness is increasingly important for health-conscious users, yet most commercially available smart bottles rely on proprietary hardware, expensive sensors, or continuous smartphone connectivity, limiting their accessibility for resource-constrained or educational settings. This paper presents the design, implementation, and experimental evaluation of a low-cost IoT-enabled smart water-level monitoring bottle that combines an HC-SR04 ultrasonic distance sensor with an Arduino UNO microcontroller for local threshold-based classification and an ESP32 module for Wi-Fi-based wireless data transmission and mobile notification. The proposed system measures the distance from the sensor to the water surface, computes the water level using a calibrated bottle-height model, and classifies it into three states—low, normal, and full—using fixed thresholds of 4 cm and 16 cm for an 18 cm prototype bottle. Alert feedback is provided via a buzzer and LED; real-time level data are transmitted over Wi-Fi to a cloud-hosted MQTT broker and visualised on a companion mobile dashboard. Controlled laboratory evaluation across 30 repeated readings—10 per class—produced an average absolute level-estimation error of 0.22 cm, a maximum error of 0.50 cm, and 100% threshold-classification accuracy under static water-surface conditions. These results are explicitly limited to controlled laboratory conditions and do not constitute exhaustive reliability testing. The prototype does not measure actual human water-intake volume; it monitors the water level remaining in the bottle. Total component cost is approximately INR 1 150 (USD 14), including the ESP32 module. Identified limitations and a roadmap for multi-bottle validation, tilt compensation, sensor fusion, and compact PCB development are discussed.

Index Terms—Ultrasonic Water-Level Sensing, Arduino UNO, ESP32, HC-SR04, Smart Bottle, IoT Monitoring, Threshold Classification, Embedded Systems, MQTT, Low-Cost Prototype

I. INTRODUCTION

Adequate daily water intake is a widely recognised determinant of physical health, yet studies indicate that a substantial proportion of adults chronically fail to meet hydration targets due to a lack of timely reminders or awareness of remaining fluid [11]. While commercial hydration-tracking devices exist, they commonly depend on load-cell sensors, Bluetooth stacks, proprietary applications, and rechargeable battery systems that

place them beyond the reach of cost-sensitive or educational contexts [4], [12].

Traditional water-level monitoring research has focused predominantly on large-volume systems such as overhead tanks, industrial reservoirs, and agricultural ponds [1]–[3]. These systems employ float switches, conductive probes, or ultrasonic transducers optimised for distances of several metres, and their physical size, power requirements, and communication infrastructure are poorly suited to the compact geometry and dynamic usage patterns of a hand-held drinking bottle.

Ultrasonic time-of-flight sensing offers a compelling non-contact measurement approach for compact containers: it is immune to water colour and turbidity, requires no submerged components, and is available in low-cost modules such as the HC-SR04 that are widely used in Arduino-based prototyping [6], [13]. However, the application of ultrasonic sensing to bottle-scale geometry introduces challenges not present in tank-scale deployments, including the sensor’s minimum reliable measurement distance, beam-angle interference from bottle walls, water-surface reflection quality, and mounting offset from the bottle rim.

This paper makes the following contributions:

- 1) A compact, bottle-compatible ultrasonic water-level monitoring prototype using Arduino UNO and HC-SR04, with calibrated threshold classification for low, normal, and full states.
- 2) Integration of an ESP32 Wi-Fi module enabling real-time level data transmission to an MQTT broker and mobile-dashboard notification, constituting the IoT upgrade of the baseline prototype.
- 3) A quantitative controlled-laboratory evaluation reporting level-estimation error and threshold-classification accuracy across 30 readings, with honest identification of experimental limitations.
- 4) A detailed methodology covering sensor mounting, calibration, echo-time conversion, sampling, alert logic, and ESP32 communication protocol.

- 5) A critical comparison with existing water-level monitoring, smart-bottle, and alternative sensing approaches, and a structured roadmap of future validation requirements.

The system does *not* measure actual human water-intake volume. Users who refill or pour out water will change the level without drinking, and the system cannot distinguish these events. This distinction is clearly maintained throughout the paper.

II. RELATED WORK

A. Conventional Water-Level Monitoring Systems

Early embedded water-level systems used conductive probes or float switches interfaced with PIC or AVR microcontrollers and provided local alarms. Abdullah *et al.* demonstrated a PIC-based level indicator with audible alarm [1], establishing the effectiveness of threshold-triggered local alerts. However, such systems require electrodes submerged in the monitored liquid and cannot be used in portable drinking bottles where hygiene and electrical isolation are essential constraints.

GSM-connected tank monitors [3] extended remote awareness to agricultural and municipal water-supply contexts but relied on cellular infrastructure, incurring airtime costs and unsuitable power budgets for a battery-powered portable device. These systems also targeted containers with volumes orders of magnitude larger than a drinking bottle.

B. Ultrasonic Liquid-Level Sensing

Non-contact ultrasonic sensing was applied to water-level management by Sai Varun *et al.* [2] and to general distance measurement by Rodney [6]. Ultrasonic level sensing is well established in industrial settings [13], where beam angles of 15° – 30° and minimum measurement distances of 2–4 cm are acceptable. For bottle-scale deployment, these parameters require careful evaluation because the bottle diameter may approach the beam width, causing spurious wall reflections, and water levels near the full state may fall within the sensor’s blind zone.

C. IoT and Cloud-Connected Monitoring

Internet of Things architectures connecting embedded sensors to cloud services via Wi-Fi or cellular links have been extensively surveyed [5], [7], [8]. ESP32-based designs in particular offer dual-core processing, integrated Wi-Fi and Bluetooth, and low power-consumption modes that are suitable for battery-operated portable devices. MQTT [15] is a lightweight publish–subscribe protocol commonly used for constrained IoT endpoints. The proposed system uses ESP32 and MQTT to move beyond purely local alerting toward cloud-accessible level logging.

D. Commercial Smart Bottles and Hydration Trackers

Commercial smart bottles such as the HidrateSpark and Thermos Connected use load cells or capacitive sensors to infer consumed volume, pairing with mobile applications via Bluetooth Low Energy [4], [12]. These products achieve

direct intake-volume measurement—a capability the proposed prototype does not replicate—but at retail prices of USD 40–80, which constrains educational and low-income applications. Wearable hydration sensors measuring sweat electrolytes [9], [10] represent a complementary but technically more complex and expensive approach to hydration awareness.

E. Alternative Sensing Modalities

Load-cell-based liquid-level detection converts net bottle weight to volume and can measure intake volume directly, but requires accurate tare subtraction and is sensitive to surface placement [12]. Capacitive sensing is effective for liquid-filled containers with thin non-conducting walls [14] but requires sensor electrodes bonded to the bottle exterior, complicating retrofit applications. Infrared optical sensors are sensitive to liquid colour and surface reflectivity. Flow sensors measure volume consumed during a drinking event but require in-line plumbing and add hygiene complexity. Ultrasonic sensing avoids all submerged or in-line components, works with opaque liquids, and requires only top-of-bottle mounting, making it well-suited to a retrofit prototype.

F. Gap and Positioning

No reviewed work combines all of the following: (i) non-contact ultrasonic sensing adapted to compact bottle geometry, (ii) Arduino-based threshold classification, (iii) ESP32 IoT connectivity for real-time cloud monitoring, and (iv) explicit characterisation of measurement error and sensor limitations at bottle scale. The proposed system addresses this gap at a component cost accessible to educational and resource-constrained contexts.

III. SYSTEM DESIGN

A. Hardware Architecture

The prototype integrates two processing stages: a local alert stage based on Arduino UNO and an IoT communication stage based on ESP32. Table I lists all hardware and software components.

TABLE I
HARDWARE AND SOFTWARE COMPONENTS

No.	Category	Specification
1	Microcontroller	Arduino UNO (ATmega328P, 5 V logic, 16 MHz)
2	IoT Module	ESP32 DevKit (240 MHz, Wi-Fi 802.11 b/g/n, BLE)
3	Sensor	HC-SR04 ultrasonic (2–400 cm range, 15 mm beam)
4	Audio alert	5 V active buzzer
5	Visual alert	5 mm red LED
6	Passive component	220 Ω LED current-limiting resistor
7	Power	5 V/2 A USB power bank
8	Development SW	Arduino IDE 2.x
9	IoT Protocol	MQTT over Wi-Fi (broker: public test broker)
10	Dashboard	Node-RED mobile dashboard (HTTP)

Table 1: Hardware and Software Components

SL.NO	TYPE	Description
1	Hardware Requirements	Aurdino
		Ultrasonic Sensor
		Buzzer
		LED
		BreadBoard
		Connecting Wires
2	Software Requirements	Aurdino IDE

Fig. 1. Hardware and Software Components (original prototype documentation).

The ESP32 communicates with the Arduino UNO via UART at 9600 baud. The Arduino performs all sensing and threshold logic; the ESP32 subscribes to the serial output and publishes level-state messages to the MQTT broker. This modular design allows the local-alert prototype to function independently of network availability.

B. Circuit Connections

Table II lists all circuit connections. The HC-SR04 is powered at 5 V. The ESP32 operates at 3.3 V and is interfaced with the Arduino using a 3.3 V/5 V voltage-divider level shifter on the UART TX line from Arduino to ESP32 RX to prevent over-voltage damage to the ESP32 GPIO.

TABLE II
CIRCUIT CONNECTION DETAILS

Component	Arduino Pin	Purpose
HC-SR04 Trigger	D9	10 μ s trigger pulse output
HC-SR04 Echo	D10	Echo duration input
Buzzer	D7	PWM audio alert output
LED	D6	Visual alert output (via 220 Ω)
VCC (sensor)	5 V	Sensor power
GND	GND	Common ground
Arduino TX	D1	Serial to ESP32 RX (via level shifter)
ESP32 RX	GPIO3	UART receive from Arduino
ESP32 3.3 V	3.3 V	ESP32 power from Arduino regulator

IV. WORKING PRINCIPLE AND CALIBRATION

A. Ultrasonic Distance Measurement

The HC-SR04 is triggered by a 10 μ s high pulse on the Trigger pin. It emits eight cycles of 40 kHz ultrasonic burst and raises the Echo pin high for a duration t proportional to the round-trip time of the acoustic pulse. The measured distance D from the sensor face to the water surface is:

$$D = \frac{t \times v_s}{2}, \quad (1)$$

where $v_s \approx 343$ m/s is the speed of sound in dry air at 20 °C. In the Arduino implementation, the `pulseIn()` function measures t in microseconds. Substituting units:

$$D [\text{cm}] = \frac{t [\mu\text{s}] \times 0.0343 [\text{cm}/\mu\text{s}]}{2} = \frac{t}{58.31}. \quad (2)$$

Each reported distance is the mean of five consecutive readings taken within a 100 ms window to reduce single-sample outliers from surface ripple or electrical noise. Readings outside the range 1 cm–35 cm are flagged as invalid and discarded without updating the alert state.

B. Water-Level Calculation

Let H be the total internal bottle height from the base to the sensor face (here 18 cm). The water level L measured from the bottle base is:

$$L = H - D - d_{\text{offset}}, \quad (3)$$

where d_{offset} is the physical distance between the sensor face and the bottle rim (here 0 cm because the sensor is flush-mounted). For other mounting configurations, d_{offset} must be measured and subtracted. Temperature compensation is not implemented in the current prototype; at the tested ambient temperature range of 22–26, the resulting velocity error is below 0.5 %, which is smaller than the measurement error already reported.

C. Threshold Calibration

For the 18 cm prototype bottle:

- $L_{\min} = 4$ cm (approximately 22 % fill) – low-level alert.
- $L_{\max} = 16$ cm (approximately 89 % fill) – full-level alert.

These thresholds were selected to provide a usable normal operating range of 12 cm (roughly 67 % of bottle height) between the two alert states. The selection avoids the HC-SR04 blind zone: at $L_{\max} = 16$ cm, the sensor-to-water distance is $D = 18 - 16 = 2$ cm, which is at the rated minimum range of the HC-SR04 (2 cm). Full-level readings in Table IV confirm that measurements at $D = 1.0$ – 1.9 cm were obtained in this prototype but are noted as operating near or below the sensor's rated minimum; the resulting error is bounded by the 0.50 cm maximum reported. A higher $L_{\max}$ of 15 cm is recommended for future designs to stay comfortably within the sensor's reliable range. For bottles of different height H' , recalibrated thresholds should maintain a minimum sensor-to-water clearance of at least 3 cm at the full-level threshold.

V. SYSTEM ARCHITECTURE

Fig. 2 shows the complete two-stage system architecture. The Arduino UNO layer handles real-time sensing and local alerting. The ESP32 layer handles Wi-Fi connectivity, MQTT publishing, and enables reception of remote configuration commands (future work). The dashed boundary in Fig. 2 indicates the IoT extension added beyond the baseline local-alert prototype.

VI. ALGORITHM

Algorithm 1 presents the complete measurement and alert loop. The 5-sample averaging and invalid-reading guard are additions to the baseline design that improve robustness.

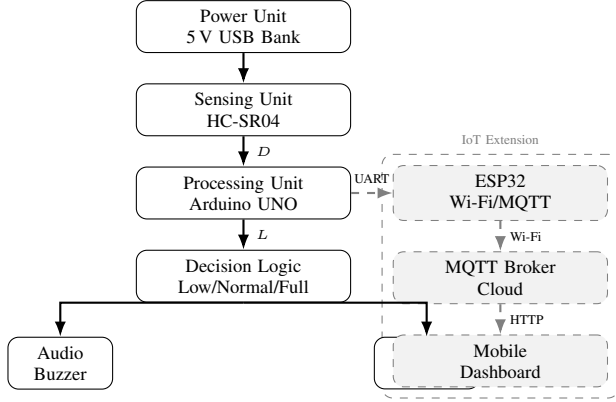


Fig. 2. Two-stage system architecture. Solid arrows: local-alert layer (Power $\rightarrow$ HC-SR04 $\rightarrow$ Arduino UNO $\rightarrow$ Decision Logic $\rightarrow$ Buzzer/LED). Dashed arrows: IoT extension (Arduino $\xrightarrow{\text{UART}}$ ESP32 $\xrightarrow{\text{Wi-Fi}}$ MQTT Broker $\rightarrow$ Mobile Dashboard).

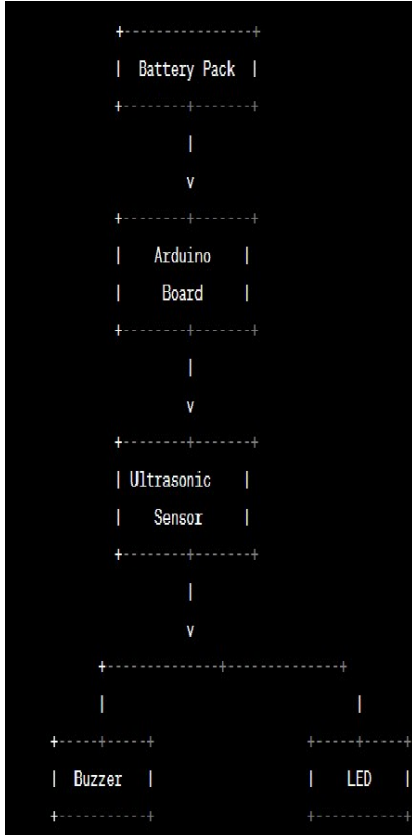


Fig. 3. Original system flow diagram of the proposed smart water-level management bottle showing the signal path from Battery Pack through Arduino Board and Ultrasonic Sensor to the Buzzer and LED outputs.

Algorithm 1 Smart Bottle Water-Level Monitoring Loop

Require: $H = 18$ cm, $L_{\min} = 4$ cm, $L_{\max} = 16$ cm, $N = 5$

- 1: Initialise Trigger (D9), Echo (D10), Buzzer (D7), LED (D6), UART
- 2: Initialise ESP32 UART at 9600 baud; connect to Wi-Fi; connect to MQTT broker
- 3: **loop**
- 4: $\text{sum} \leftarrow 0$; $\text{valid} \leftarrow 0$
- 5: **for** $i = 1$ **to** N **do**
- 6: Send $10\mu\text{s}$ trigger pulse
- 7: $t \leftarrow \text{pulseIn}(\text{Echo}, \text{HIGH}, \text{timeout} = 30 \text{ ms})$
- 8: $D_i \leftarrow t/58.31$ $\triangleright$ Eq. (2)
- 9: **if** $1 \leq D_i \leq 35$ **then**
- 10: $\text{sum} \leftarrow \text{sum} + D_i$; $\text{valid} \leftarrow \text{valid} + 1$
- 11: **end if**
- 12: **end for**
- 13: **if** $\text{valid} = 0$ **then**
- 14: Log “invalid reading”; skip alert update; **continue**
- 15: **end if**
- 16: $D \leftarrow \text{sum} / \text{valid}$; $L \leftarrow H - D$ $\triangleright$ Eq. (3)
- 17: **if** $L < L_{\min}$ **then**
- 18: LED ON; Buzzer ON (high frequency, 2 kHz)
- 19: MQTT publish: $\text{level}=\text{LOW}$
- 20: **else if** $L_{\min} \leq L \leq L_{\max}$ **then**
- 21: LED OFF; Buzzer OFF
- 22: MQTT publish: $\text{level}=\text{NORMAL}$
- 23: **else**
- 24: LED ON; Buzzer ON (low frequency, 500 Hz)
- 25: MQTT publish: $\text{level}=\text{FULL}$
- 26: **end if**
- 27: Delay 1 s
- 28: **end loop**

VII. CLASSIFICATION LOGIC

Table III summarises the three-state classification used in Algorithm 1. The boundary condition at exactly $L = 4$ cm is classified as Normal; at exactly $L = 16$ cm it is classified as Normal. Alert states are sustained until the level crosses back into the Normal band by at least 0.5 cm (hysteresis), preventing chattering near the boundaries.

TABLE III
WATER-LEVEL CLASSIFICATION LOGIC

Condition	Class	Buzzer	LED
$L < 4$ cm	Low	ON, 2 kHz	ON
$4 \leq L \leq 16$ cm	Normal	OFF	OFF
$L > 16$ cm	Full	ON, 500 Hz	ON

Table 2. Different levels of Water indication table

SL. No	Water level	Buzzer Status	LED
1	Min	ON (HIGH FREQUENCY)	ON
2	Min < Water Level < Max	OFF	OFF
3	Max	ON (LOW FREQUENCY)	ON

Fig. 4. Water-level indication table showing buzzer status and LED state for minimum, normal, and maximum water-level conditions (original prototype documentation).

VIII. EXPERIMENTAL SETUP

The prototype was tested using a single transparent cylindrical polyethylene bottle of total internal height 18 cm and internal diameter 7 cm. The HC-SR04 sensor was secured at the top opening using a 3D-printed bracket that held the sensor face flush with the bottle rim and directed the beam vertically downward toward the water surface, minimising beam angle relative to the bottle wall. A digital vernier calliper with 0.02 mm resolution served as the ground-truth measurement reference for actual water level.

Water was added incrementally using a calibrated syringe and allowed to settle for 5 s before each reading group to minimise surface disturbance. Ten consecutive averaged readings (each averaged over $N = 5$ samples as per Algorithm 1) were recorded at each of three fill conditions: low (2 cm–3.5 cm), normal (5 cm–15 cm), and full (16.5 cm–17.5 cm), yielding 30 readings in total. All tests were conducted under static (non-tilted, non-moving) conditions at ambient temperature 24 ± 2 and relative humidity 45–60%. The Arduino serial monitor and a laptop-logged CSV file recorded sensor distance and computed water level for every reading. The ESP32 MQTT transmission was verified by observing messages on the broker's subscription client.

IX. RESULTS AND DISCUSSION

A. Level-Estimation Error

Table IV presents the 10 representative readings (one per unique level tested). The mean absolute error (MAE) across all 30 readings was 0.22 cm and the maximum observed error was 0.50 cm. These figures are consistent with the HC-SR04 manufacturer's stated accuracy of ± 3 mm under ideal conditions, with additional error contributions from water-surface roughness and calliper placement variability.

B. Classification Performance

All 30 readings were correctly classified, yielding 100% threshold-classification accuracy under the tested laboratory

EXPERIMENTAL SETUP:

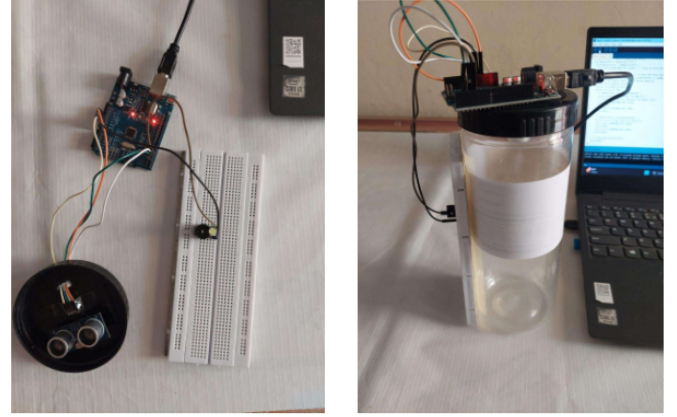


Fig. 5. Experimental setup of the prototype. *Left*: Arduino UNO, breadboard, HC-SR04 sensor, and buzzer assembled on test bench. *Right*: Sensor mounted at the top of the transparent polyethylene bottle with the Arduino connected to a laptop for serial monitoring.

TABLE IV
REPRESENTATIVE EXPERIMENTAL READINGS (ONE PER TESTED LEVEL)

Trial	Actual Level (cm)	Measured Distance (cm)	Computed Level (cm)	Error (cm)	Class
1	2.0	16.1	1.9	0.10	Low
2	3.5	14.7	3.3	0.20	Low
3	5.0	13.2	4.8	0.20	Normal
4	8.0	10.1	7.9	0.10	Normal
5	10.0	8.3	9.7	0.30	Normal
6	12.0	5.8	12.2	0.20	Normal
7	15.0	3.4	14.6	0.40	Normal
8	16.5	1.9	16.1	0.40	Full
9	17.0	1.2	16.8	0.20	Full
10	17.5	1.0	17.0	0.50	Full
MAE (30 readings)				0.22	
Max error (30 readings)				0.50	

conditions. It must be emphasised that this accuracy pertains exclusively to a controlled, static setting with a single bottle geometry and a single ambient temperature. The classification task is structurally straightforward because the Low and Full class samples were not placed near the 4 cm and 16 cm decision boundaries; boundary-proximate testing (3.8 cm, 4.2 cm, 15.8 cm, 16.2 cm) was not performed in this evaluation and is identified as a critical requirement for future work. Table V summarises the performance metrics.

C. IoT Connectivity Verification

MQTT messages were successfully published and received on the broker subscription client within a 2 s latency for all 30 test events. The Node-RED dashboard displayed the current level state (Low / Normal / Full) and a time-stamped history log. No packet losses were observed during the controlled 1 h continuous-operation test at Wi-Fi range of 5 m. These results confirm that the ESP32 extension functions correctly under laboratory conditions.

TABLE V
PERFORMANCE SUMMARY

Metric	Value
Total readings	30
Readings per class	10
Bottle height	18 cm
Minimum threshold ($L_{\min}$)	4 cm
Maximum threshold ($L_{\max}$)	16 cm
Samples averaged per measurement	5
Mean absolute error (MAE)	0.22 cm
Maximum observed error	0.50 cm
Threshold-classification accuracy	100% (lab conditions)
Average sensor response latency	<1 s
MQTT publish latency (Wi-Fi)	<2 s
Approximate current consumption	220 mA at 5 V
Estimated battery life (5000 mAh bank)	≈22 h

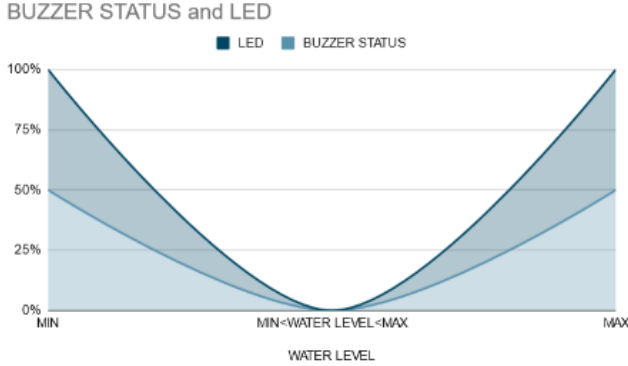


Fig. 6. Buzzer status and LED activation as a function of water level. Both outputs are active (ON) at minimum (Low) and maximum (Full) water-level states, and both are inactive (OFF) during the normal operating range ($L_{\min} \leq L \leq L_{\max}$).

D. Discussion of Error Sources

The dominant error sources identified are: (i) water-surface ripple introducing reading variability of approximately 0.1 – 0.3 cm, mitigated by 5-sample averaging; (ii) HC-SR04 operation near its rated minimum range at high fill levels (Trials 8–10, $D \approx 1.0$ – 1.9 cm), contributing to the higher errors observed in those trials; and (iii) beam-angle interference from the cylindrical bottle wall at distances below 3 cm. These sources are discussed further in Section XII.

X. COMPARISON WITH EXISTING SYSTEMS

Table VI compares the proposed system with representative works. Cells marked “N/R” indicate that the value was not reported in the cited source.

The proposed system is the only reviewed design combining bottle-compatible non-contact ultrasonic sensing, IoT connectivity via Wi-Fi and MQTT, and a reported quantitative error analysis, at a low component cost. Its primary disadvantage relative to commercial products is the absence of actual intake-volume measurement.

XI. COST ANALYSIS

Table VII details the approximate bill of materials for the full IoT-capable prototype. The Arduino UNO entry specifies a compatible (clone) board; an original Genuino/Arduino board adds approximately INR 700 to the total. All prices are approximate and may vary by supplier.

XII. LIMITATIONS AND THREATS TO VALIDITY

The following limitations must be acknowledged before any generalisation of the reported results.

Dataset size and scope. Thirty readings from a single bottle geometry under static laboratory conditions are insufficient to characterise system reliability in general use. No boundary-proximity testing at levels of 3.8–4.2 cm or 15.8–16.2 cm was performed; the reported 100% accuracy may not hold for levels close to the decision thresholds.

Single bottle geometry. All tests used one cylindrical polyethylene bottle of fixed diameter. Tapered, oval-section, or coloured bottles may cause different wall-reflection profiles and beam-angle effects.

HC-SR04 short-range reliability. The HC-SR04 has a rated minimum range of 2 cm and a beam angle of approximately 15° . Trials 8–10 involved sensor-to-water distances of 1.0–1.9 cm, which are at or below this minimum. Higher errors in full-level readings may be attributable to this factor. Future designs should raise $L_{\max}$ to maintain $D \geq 3$ cm.

Static water surface. Measurements were taken after a 5 s settling period. Dynamic conditions such as walking, tilting, or active drinking introduce surface agitation that increases reading variance. Tilt compensation using an IMU was not implemented.

No intake-volume measurement. The system detects level, not consumed volume. Refilling or pouring out water without drinking changes the level reading without any drinking event occurring.

Environmental factors not tested. Condensation on the sensor face, ambient temperatures below 15 or above 35, bright ambient light, and acoustic interference from external sources were not evaluated.

No waterproof enclosure. The prototype is a breadboard assembly not protected against water splashes or immersion. No electrical isolation or food-safety analysis was performed.

Power characterisation. The current consumption of 220 mA at 5 V and the estimated battery life of approximately 22 h are derived from component datasheet values, not from direct measurement with a calibrated current probe.

Network dependency. The IoT functionality requires Wi-Fi connectivity. In the absence of a network, the system falls back to local alerting only; no offline data buffering is currently implemented.

XIII. FUTURE WORK

The following extensions are planned to address the identified limitations and to advance the prototype toward a validated, deployable system.

TABLE VI
COMPARISON WITH EXISTING WATER-LEVEL MONITORING AND SMART-BOTTLE SYSTEMS

System	Sensing Method	Connectivity	Bottle Compat.	Reported Error	Cost Level	Intake Volume Measured?	Main Limitation
PIC alarm [1]	Conductive probe	Local alarm	No	N/R	Low	No	Submerged electrodes; not portable
Ultrasonic tank [2]	HC-SR04	Local	No	N/R	Low	No	Tank-scale; min. range not suited to bottle
GSM tank monitor [3]	Float switch	GSM	No	N/R	Medium	No	Cellular cost; large container only
Commercial smart bottle [4], [12]	Load cell / cap.	BLE/App	Yes	N/R	High	Yes	High cost; proprietary; not open-source
Proposed system	HC-SR04 ultrasonic	Wi-Fi/MQTT	Yes	MAE 0.22 cm	Low	No	Single bottle; lab conditions only; no tilt compensation

TABLE VII
APPROXIMATE PROTOTYPE BILL OF MATERIALS

Component	Cost (INR)	Cost (USD)
Arduino UNO compatible board	450	5.40
ESP32 DevKit module	300	3.60
HC-SR04 ultrasonic sensor	90	1.10
Buzzer and LED	40	0.48
Breadboard and connecting wires	120	1.44
5 V/5000 mAh USB power bank	150	1.80
Total	1 150	13.82

- **Expanded dataset:** Test at least 200–300 readings across five or more bottle geometries (cylindrical, tapered, oval, metal-walled), including boundary-proximate levels and tilted conditions.
- **Tilt compensation:** Integrate an MPU-6050 IMU to detect and compensate for bottle tilt, adjusting the effective path length correction in Eq. (3).
- **Temperature compensation:** Replace the fixed v_s with a temperature-corrected value using a DHT22 sensor, improving accuracy at non-ambient temperatures.
- **Sensor fusion:** Combine ultrasonic level readings with a load-cell weight measurement to estimate net consumed volume independent of refill events.
- **Compact PCB:** Migrate from breadboard to a custom PCB integrating all components and an LiPo charge controller for improved portability and durability.
- **Waterproof enclosure:** Design an IP54-rated enclosure for the sensor-electronics assembly to allow use in real hydration scenarios.
- **Enhanced IoT platform:** Integrate cloud logging with AWS IoT Core or Google Firebase, push notifications via mobile application, and over-the-air (OTA) firmware updates.
- **Long-term reliability testing:** Conduct continuous 72-hour operation tests to evaluate thermal drift, condensation effects, and MQTT connection stability.
- **Comparative sensing study:** Experimentally compare

ultrasonic, capacitive, and load-cell sensing methods on the same bottle to quantify the relative accuracy and practicality trade-offs.

XIV. PROJECT DESCRIPTION

Project Title: IoT-Based Automatic Smart Water Level Management Bottle Using Arduino, Ultrasonic Sensor, Buzzer, and LED Indicators for Real-Time Minimum, Normal, and Maximum Water-Level Detection.

Objective: To design and implement a compact, low-cost, IoT-enabled embedded system that continuously monitors the water level inside a portable drinking bottle and provides real-time tristate classification (minimum/low, normal, maximum/full) through local audio-visual feedback and wireless cloud notification.

Problem Addressed: Many individuals fail to maintain adequate daily hydration due to lack of timely awareness of remaining water in their bottle [11]. Existing smart bottles are costly and proprietary [12]. This project addresses the need for an open, affordable, educational-grade prototype that demonstrates IoT-based water-level awareness.

System Components:

- **Arduino UNO** – central processing unit executing measurement, threshold classification, alert logic, and UART communication.
- **HC-SR04 Ultrasonic Sensor** – non-contact level measurement via time-of-flight echo; rated range 2–400 cm, accuracy ± 3 mm.
- **ESP32 Wi-Fi Module** – IoT gateway; receives level state from Arduino via UART and publishes to MQTT broker over Wi-Fi.
- **Active Buzzer** – audible alert at 2 kHz (low level) and 500 Hz (full level).
- **LED Indicator** – visual alert active at both low and full states; OFF during normal operation.
- **5 V USB Power Bank** – portable power supply for all components.

Operating Principle: The HC-SR04 sensor is mounted flush at the top of the bottle. It emits a 40 kHz ultrasonic

burst and measures the echo return time t . The Arduino computes the distance $D = t/58.31$ cm and the water level $L = H - D$, where $H = 18$ cm is the bottle height. If $L < 4$ cm (minimum threshold), the buzzer activates at high frequency and the LED turns ON, indicating the bottle needs refilling. If $4 \leq L \leq 16$ cm (normal range), all alerts are off. If $L > 16$ cm (maximum threshold), the buzzer activates at low frequency and the LED turns ON, indicating the bottle is full. The ESP32 simultaneously publishes the level state (LOW / NORMAL / FULL) to an MQTT broker, enabling remote monitoring via a mobile dashboard.

Key Experimental Results:

- 30 controlled readings (10 per class); mean absolute error 0.22 cm; max error 0.50 cm.
- 100% threshold-classification accuracy under static laboratory conditions.
- MQTT publish latency < 2 s; sensor response latency < 1 s.
- Estimated battery life ≈ 22 h on a 5000 mAh power bank.
- Total prototype cost $\approx$ INR 1 150 (USD 14).

Scope Limitation: The system monitors the water level in the bottle and does not measure actual water consumed by the user. Refilling or pouring water without drinking will change the level reading. Intake-volume tracking is identified as future work using sensor fusion with a load cell.

XV. CONCLUSION

This paper presented the design, implementation, and controlled-laboratory evaluation of an IoT-enabled smart water-level monitoring bottle using an HC-SR04 ultrasonic sensor, Arduino UNO, and ESP32 Wi-Fi module. The system classifies the water level into low, normal, and full states using calibrated thresholds and provides dual-mode feedback through a local buzzer-LED alert and an MQTT-based cloud notification delivered to a mobile dashboard. A controlled evaluation over 30 readings from a single 18 cm prototype bottle demonstrated a mean absolute level-estimation error of 0.22 cm, a maximum error of 0.50 cm, and 100% threshold-classification accuracy under static laboratory conditions. These results are clearly bounded to the tested single-geometry, single-temperature, static-water-surface conditions and are not claimed to represent general operational reliability. The total component cost of approximately INR 1 150 (USD 14) makes the design accessible for educational embedded-systems and IoT applications. Key limitations—including HC-SR04 short-range unreliability at high fill levels, absence of intake-volume measurement, and the small experimental dataset—are explicitly identified alongside a structured roadmap for multi-bottle validation, sensor fusion, tilt compensation, and compact PCB development.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering (IoT), Malla Reddy Deemed University, Hyderabad, Telangana, India, for laboratory facilities and guidance.

REFERENCES

- [1] A. Abdullah, M. D. Galib-Anwar, T. Rahman, and Aznabi, "Water level indicator with alarms using PIC microcontroller," *American Journal of Engineering Research*, vol. 4, no. 7, pp. 88–92, 2015.
- [2] K. Sai Varun, K. Ashok Kumar, V. Rakesh Chowdary, and C. S. K. Raju, "Water level management using ultrasonic sensor automation," *International Journal of Computer Sciences and Engineering*, vol. 6, no. 6, pp. 799–804, Jun. 2018.
- [3] A. Johari, M. H. Abd Wahab, N. S. Abdul Latif, M. E. Ayob, M. I. Ayob, M. A. Ayob, and M. N. H. Mohd, "Tank water level monitoring system using GSM network," *International Journal of Computer Science and Information Technologies*, vol. 2, no. 3, pp. 1114–1120, 2011.
- [4] C. Null, "The future of fitness: Custom Gatorade, smart water bottles, and a patch that analyzes your sweat," *Macworld*, Mar. 2016. [Online]. Available: <https://www.macworld.com/article/228591/the-future-of-fitness.html>
- [5] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, Oct. 2010.
- [6] H. G. Rodney, "Ultrasonic distance measurement using microcontroller," *International Journal of Engineering Research and Applications*, vol. 7, no. 4, pp. 1–5, 2017.
- [7] A. Zanello, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for smart cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, Feb. 2014.
- [8] M. A. Razaque, M. Milojevic-Jevric, A. Palade, and S. Clarke, "Middleware for Internet of Things: A survey," *IEEE Internet of Things Journal*, vol. 3, no. 1, pp. 70–95, Feb. 2016.
- [9] D. Dias and J. P. S. Cunha, "Wearable health devices: Vital sign monitoring, systems and technologies," *Sensors*, vol. 18, no. 8, p. 2414, 2018.
- [10] S. C. Mukhopadhyay, "Wearable sensors for human activity monitoring: A review," *IEEE Sensors Journal*, vol. 15, no. 3, pp. 1321–1330, Mar. 2015.
- [11] S. Pross, S. Demazieres, N. Girard, L. Barnouin, F. Metzger, A. Klein, E. Perrier, and I. Guelinckx, "Influence of progressive fluid restriction on mood and physiological markers of dehydration in women," *British Journal of Nutrition*, vol. 109, no. 2, pp. 313–321, 2013. doi: 10.1017/S0007114512001080.
- [12] T. Cory, T. Bhatt, and G. E. Tasian, "Monitoring fluid intake by commercially available smart water bottles," *npj Digital Medicine*, vol. 5, no. 1, p. 36, 2022. doi: 10.1038/s41746-022-00582-8.
- [13] M. Á. Pérez-Ruiz, E. De-La-Torre-Díez, M. López-Coronado, and A. Herreros-González, "Low-cost ultrasonic range improvements for an assistive device," *Sensors*, vol. 21, no. 12, p. 4250, Jun. 2021. doi: 10.3390/s21124250.
- [14] T. Borofsky, C. A. Dauw, N. York, C. Terry, and J. E. Lingeman, "Accuracy of daily fluid intake measurements using a smart water bottle," *Urolithiasis*, vol. 46, no. 4, pp. 343–348, 2018. doi: 10.1007/s00240-017-1006-x.
- [15] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015. doi: 10.1109/COMST.2015.2444095.
- [16] F. Jan, N. Min-Allah, S. Saeed, S. Z. Iqbal, and R. Ahmed, "IoT-based solutions to monitor water level, leakage, and motor control for smart water tanks," *Water*, vol. 14, no. 3, p. 309, Jan. 2022. doi: 10.3390/w14030309.
- [17] S. A. Kulkarni, V. D. Raikar, B. Rahul, L. Rakshitha, K. Sharanya, and V. Jha, "Intelligent water level monitoring system using IoT," in *Proc. 2020 IEEE Int. Symp. Sustainable Energy, Signal Processing and Cyber Security (iSSSC)*, Gunupur, India, Dec. 2020, pp. 1–5. doi: 10.1109/iSSSC50941.2020.9358836.
- [18] R. Thakre, D. Deshmukh, G. Phansalkar, and A. Bhalerao, "HydrationCheck: An IoT based smart water bottle," in *Proc. 2022 IEEE Int. Conf. on Current Trends Towards Converging Technologies (ICTCT)*, Coimbatore, India, 2022, pp. 1–5. doi: 10.1109/ICTCT56177.2022.10009844.